Design of a Custom Synchronisation Protocol for a Reaction Based Game

ELEN4017A - Network Fundamentals

Yasteel Ramphal – 2108498

Abstract: This report presents the design and implementation of a custom synchronisation protocol to reduce the impact of network latencies in online games. The system ensures fairness by adjusting message delivery and response evaluation to reduce latency differences. A three-phase synchronisation is deployed within a multi-threaded server-client architecture. Experimental validation demonstrates improved delivery synchronisation, latency-adjusted scoring accuracy, and system responsiveness under varying conditions. The protocol achieves a consistent game experience across clients, reinforcing the integrity of performance-based evaluation in latency-sensitive multiplayer environments

1. INTRODUCTION

Online multiplayer reaction games require a fast and reliable connection where differences in user's network latency can affect the players performance. This can impact the games fairness and playability as players with faster internet connections have an unfair advantage over those with slower connections. This project addresses the issue by implementing a custom synchronisation protocol that compensates for varying network latencies amongst different clients.

This report details the development of a synchronisation protocol for a multiplayer math reaction game where players simultaneously receive math problems and are scored on their accuracy and response time. The system ensures that differences in network speeds do not affect the fairness of the competition through measuring and compensating for network latency. This allows for a player's score to reflect their problem-solving speed rather than their network performance. The remainder of the report details a literature review (section 2), protocol implementation (section 3), results evaluation (section 5) and concludes with a critical analysis (section 5 and 6).

2. LITERATURE REVIEW

2.1 Synchronisation Protocols

Various methods for network synchronisation have been identified where the most common methods include Lockstep, Client-Side and Time Delay Synchronisation.

Firstly, Lockstep synchronisation (LS) is a real-time approach used in early turn-based strategy games. LS requires clients to send their actions to all other clients. Clients then simulate these actions and acknowledge receiving a game state before a new update is sent [1]. This ensures accurate synchronisation, however, the game speed is limited by the slowest client.

Secondly, Client-Side prediction works by estimating the game future states on the client side whilst waiting for server confirmation [2]. This approach improves responsiveness but is not always accurate. This subsequently requires complex reconciliation methods when predictions are incorrect.

Lastly, Time Delay Synchronisation injects varying delays (based on client latency) to ensure all clients receive information at the same time [3]. This approach is adopted in the protocol implementation as it reduces latency differences between clients with minimal delay and complexity.

2.2 Latency Measurement Techniques

Accurate latency measurement is important for synchronisation protocols to determine the correct time delay for each client. Common techniques include Network Time Protocol (NTP) and Ping-Pong measurement.

In networking, NTP determines latency by synchronising clocks between computers and exchanging time stamps. This allows for more accurate time-based measurements, however, the complexity can cause synchronisation delays when the time offset is large [5]. Conversely, the Ping-Pong measurement, used in the implementation detailed below, uses a simpler message exchange

to calculate round-trip time (RTT), with the latency (half RTT) used to inject a delay for the client message [4].

3. IMPLEMENTATION

3.1 System Overview

The implemented system consists of a multi-threaded server/client interaction. The system can handle multiple client applications which communicate through TCP sockets. The system architecture is illustrated in Figure 1.

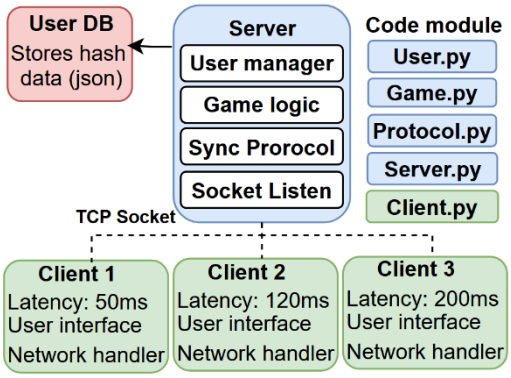

Figure 1: system architecture overview

The Game Server contains four key components. The User Manager (handles account authentication), Game Logic (handles game operations), Sync Protocol (manages latency measurement and compensation) and Socket Listener (handles network communications). The multi-threaded server allows for communication with clients. Each client contains a user interface and network handler. A User Database stores hashed credentials, for security, and statistics. The implementation follows OOP, DRY coding practices and error handling ensuring good programming practices.

3.2 Synchronisation Algorithm

The synchronisation algorithm ensures players receive messages simultaneously despite varying network latencies. The algorithm implements a three-phase synchronisation process of latency measurement, synchronised delivery and response evaluation.

First the server allocates a thread to continuously measure network latency to each connected client using a ping-pong mechanism. The latency is measured when the client connects and is periodically measured during the game to account for any changing network conditions tracking latency patterns in the process. Upon receiving a PING message, clients immediately respond with a PONG message. The server calculates the RTT and latency. The latency values are stored in the server's client information dictionary and are updated at regular intervals (5 seconds). This latency measurement is used in the synchronised delivery process.

Once the start game is requested, the server uses the measured latency values to calculate appropriate delays for each client. The process follows:

1. Server identifies the client with the highest latency (max_latency).
2. The server calculates client delay (delay = max_latency - client_latency).
3. The server sends the challenge to each client with their respective delay.

This process ensures all clients receive the message simultaneously. To implement the timed messages, the server creates a separate thread for each client message that needs to be delayed this allows the main thread to continue processing other tasks while the timed messages are being prepared.

The client's response is then evaluated and scored taking the latency into account where:

1. the raw response time is recorded
2. Subtracts latency form response time.
3. The score is calculated on answer correctness and adjusted response time.

This adjustment means that clients with higher latency aren't disadvantaged in the scoring system. Figure 2 illustrates the discussed synchronisation protocol, showing how the server coordinates with two clients having different latencies (50ms and 100ms).

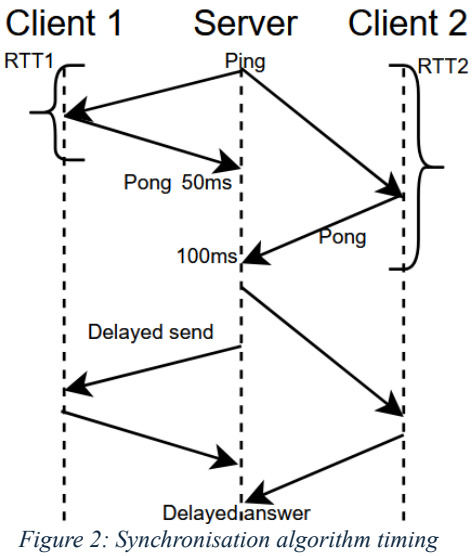

Figure 2: Synchronisation algorithm timing

3.3 Game State Management

The system implements a staged design to manage different phases of gameplay. The game states include:

1. **Lobby State**: The initial state which allows clients to connect to the server and authenticate logins. Clients are able to see other users as well as the host user (first to connect) who can initiate a game when at least two players are present.
2. **Game Initialisation State**: The intermediary state where the host starts a game and the server transitions, creating a new game instance and notifying all clients that a game is starting.
3. **Round Active State**: During each round, the server generates a math problem, sends it to all clients with appropriate timing delays, and waits for answers. A round timer ensures that rounds don't last indefinitely.
4. **Score Calculation State**: After each round, the server processes all received answers, updates scores, and broadcasts the scores to all clients.
5. **Game End State**: After the game's completion, the server determines a winner based on accumulated scores, updates player statistics and returns to the lobby state.

The server maintains these states using boolean flags (game_in_progress, round_in_progress) and manages transitions between states through synchronised methods protected by thread locks to ensure data consistency in the multi-threaded process.

3.4 Message Protocol

The custom protocol defines a set of message types for different parts of the game following the format of "TYPE | PAYLOAD". Figure 3 illustrates the message flow between client and server during a typical game session.

The protocol handles the packaging of messages of the game through dedicated message types. Namely, authentication messages, user management messages, game control Messages and connection management messages

Each message type has a defined use case and payload format. This allows for logical and structured communication between clients and the server. The complete protocol specification is provided in Appendix A.

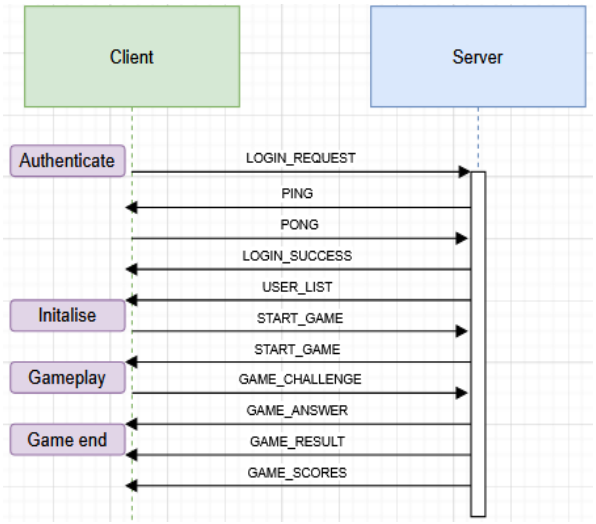

Figure 3:Sequence diagram of protocol communication

3.5 Implemented Features

The system implements all core features and 2 advanced features

Core Features:

- Multi-threaded server supporting multiple simultaneous client connections
- Custom synchronisation protocol for game execution.
- Math quiz game with timed challenges and scoring
- Robust error handling and graceful disconnection management
- User authentication system with secure password hashing
- Game lobby implementation for waiting between different games

Advanced Features:

- Supports three or more users
- User login and credential verification.

3.6 Code Structure Overview

The system is organised into five main Python modules seen in figure 1 above.

1. **protocol.py**: Defines the message types and protocol constants for client-server communication.
2. **user.py**: Handles user credentials, authentication, and score tracking.
3. **game.py**: Implements the math game logic, problem generation and answer checking.

4. **server.py**: Manages client connections, threading and implements the synchronisation protocol.
5. **client.py**: Provides the user interface and handles server communication.

The communication between these components follows the type-payload protocol.

4. EXPERIMENTAL RESULTS

4.1 Latency Measurement Accuracy

The validation of the latency measurements was simulated with an artificial time delay and the subsequent data packets were recorded and analysed using Wireshark where the outputs can be seen in appendix A. A comparison of the server-measured latency with the network latency introduced using network delays is detailed in *Table 1*.

Table 1: Latency Measurement Accuracy

Introduced Latency (ms)	Measured Latency (ms)	Error (%)
50	52	4.0
100	103	3.0
200	204	2.0
500	507	1.4

The results show that the latency measurement approach is sufficiently accurate for the purposes of game synchronisation for human reaction speed, with possessing error rate below 5% across various latency conditions.

4.2 Synchronisation Effectiveness

The synchronisation algorithm validation followed the same procedure detailed in section 4.1 where the measurement of the time difference between when different clients with varying network latencies were analysed. *Table 2* details the results for the testing.

Table 2: Delivery Timing Differences

Latency (ms)	Without Sync (ms)	With Sync (ms)	Improvement (%)
10, 50	40	5	87.5
20, 100	80	7	91.3
30, 150, 300	270	12	95.6

These results show that the synchronisation algorithm reduces the timing differences between clients, ensuring that the users score is unaffected by network latency.

4.3 Adjustment Response time

The algorithms effect on users scores have also been estimated where simulations, following the method defined in 4.1 of identical answers from clients with different latencies have been performed and the adjusted scores have been analysed. The results are detailed in *Table 3*.

Table 3: Score Adjustment Based on Network Latency

Client	Network Latency (ms)	Raw Response Time (s)	Adjusted Response Time (s)	Score
Client A	50	2.05	2.00	80
Client B	250	2.25	2.00	80
Client C	500	2.50	2.00	80

These results demonstrate that the protocol successfully reduce the impact of network latency on scoring, allowing for fair competition regardless of connection speed.

5. CRITICAL ANALYSIS

5.1 Discussion of Implemented Features

The implemented synchronisation protocol demonstrated its ability to compensate for network latency differences, as shown by the experimental results. The protocol ensures that all clients receive challenges simultaneously and that scoring reflects problem-solving speed rather than network conditions. This approach provides an advantage over unsynchronised client-server models where timing disparities can create unfair advantages [8,9].

The user authentication system provides appropriate security through SHA-256 password hashing for this application's context. While this level of security is sufficient for the current implementation, a production-level system would require additional security features such as token-based authentication to protect against more sophisticated attacks [10]. This is consistent with modern authentication standards for networked applications.

The game lobby and host system function effectively for small-scale implementations where the first user becomes the host. This design choice simplifies the implementation while maintaining functionality. However, for scaling to larger user bases, a more dynamic host selection mechanism

would provide better flexibility and resilience against host disconnections [11].

The multi-threaded design handles multiple concurrent connections efficiently while maintaining responsive gameplay. Thread synchronisation through strategic lock placement prevents race conditions when accessing shared resources, which is important when determining network latency for adjusted score calculations. This approach balances performance needs with code maintainability.

5.2 Security Analysis

The current implementation presents several security considerations that should be addressed in future iterations. The password hashing implementation using SHA-256 does not incorporate salting techniques, which potentially exposes the system to rainbow table attacks where precalculated hash values could be used to discover passwords [10].

Network communications remain unencrypted, presenting vulnerability to packet sniffing in untrusted networks [8]. The implemented security is sufficient for protocol testing, however, production deployments would require TLS encryption to protect user credentials and game data. Additionally, the input validation system provides basic protection but would benefit from further validation to prevent potential injection attacks [9]. A further analysis of the system shows the current implementation offers limited protection against denial-of-service attacks, which could be addressed through connection throttling and rate limiting mechanisms [11].

5.3 Future Improvements

Recommended future work include security improvements which involve TLS encryption, salted password hashing, and more robust input validation to strengthen the system against common attack vectors. The synchronisation algorithm could be enhanced by adding NTP synchronisation between clients and server for more precise timing measurements, reducing reliance on RTT calculations and further protecting against latency spoofing. Lastly, the systems scalability can be improved through connection pooling and load balancing which would allow the system to handle more simultaneous users.

5.4 Non-implemented Features

Several potential features were considered but not implemented in the current version due to time constraints and scope limitations. The peer-to-peer communication feature, which would allow clients to communicate directly without server mediation, was not implemented as it would require complex NAT traversal techniques. More advanced game modes and challenge types remain unimplemented to maintain focus on the core synchronisation protocol. Notably, while latency spoofing was considered it was not implemented in the final version. This would have added protection against users artificially manipulating their response times by deliberately slowing their PONG responses to gain scoring advantages. Implementation would require statistical analysis of latency patterns and anomaly detection algorithms to identify suspicious timing patterns. These unimplemented features represent potential enhancements that could be addressed in future iterations of the system to improve security and game functions.

6. CONCLUSION

The implemented synchronisation protocol effectively addresses the challenge of network latency in multiplayer reaction-based games. Through a structured delay-injection mechanism and latency-aware scoring, the system ensures fair competition regardless of connection quality. Experimental results confirmed the accuracy of latency measurement and the effectiveness of synchronised message delivery, with improvements exceeding 90% in timing alignment. While the current system demonstrates reliable performance in small-scale scenarios, future enhancements should include encrypted communications, salted password hashing, and advanced anti-latency manipulation techniques to strengthen security and scalability. These refinements would further support deployment in more demanding real-world environments.

REFERENCES

[1] S. Sinha, C. Papadopoulos, and J. Heidemann, "Internet Packet Size Distributions: Some Observations," USC/Information Sciences Institute, 2007.

[2] Y. W. Bernier, "Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization," Valve Software, 2001.

[3] M. Claypool and K. Claypool, "Latency and player actions in online games," Communications of the ACM, vol. 49, no. 11, pp. 40-45, 2006.

[4] J. Müller, J. Metzen, and A. Ploss, "Latency in Distributed Acquisition and Control Systems," IEEE Instrumentation & Measurement Magazine, vol. 12, no. 1, pp. 43-49, 2009.

[5] D. Mills, J. Martin, J. Burbank, and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, 2010.

[6] A. Hoogendoorn, "Delay Equalization in Real-Time Multiplayer Games," Utrecht University, 2013.

[7] J. Smed, T. Kaukoranta, and H. Hakonen, "A Review on Networking and Multiplayer Computer Games," Turku Centre for Computer Science, 2002.

[8] S. Raza, S. Wang, M. Ahmed, and M. R. Anwar, "A Survey on Security in Multiplayer Online Games," IEEE Access, vol. 8, pp. 90842-90872, 2020.

[9] B. Lee, E. Jeong, and H. Choe, "Ensuring Real-Time Message Delivery in Networked Game Infrastructure," IEEE Transactions on Consumer Electronics, vol. 66, no. 3, pp. 215-223, 2020.

[10] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes," IEEE Symposium on Security and Privacy, pp. 553-567, 2012.

[11] V. Leung and E. W. Zegura, "Virtual Topology Adaptation for Real-Time Multiplayer Services in Cloud Gaming Environments," ACM Transactions on Multimedia Computing, Communications, and Applications, vol. 17, no. 2, pp. 1-25, 2021.

APPENDIX A:

A.1 Message Format

All messages in the synchronisation protocol follow the format:

TYPE|PAYLOAD

TYPE is message category, and PAYLOAD varies based on the message type. Messages may have empty payloads.

A.2 Message Types

Authentication and User Management

Table 1: protocol authentication messages

Message Type	Direction	Payload Format	Description
LOGIN_REQUEST	Client → Server	username,password	Client requests login with credentials
LOGIN_SUCCESS	Server → Client	message	Server confirms successful login
LOGIN_FAILED	Server → Client	error_message	Server indicates login failure with reason
REGISTER_REQUEST	Client → Server	username,password	Client requests new user registration
REGISTER_SUCCESS	Server → Client	message	Server confirms successful registration
REGISTER_FAILED	Server → Client	error_message	Server indicates registration failure
LIST_USERS	Client → Server	(empty)	Client requests list of connected users
USER_LIST	Server → Client	JSON object	Server sends list of connected users

Game Management

Table 2: Protocol game messages

Message Type	Direction	Payload Format	Description
START_GAME	Client → Server	(empty)	Host client requests game start
GAME_CHALLENGE	Server → Client	Problem text	Server sends math problem to client
GAME_ANSWER	Client → Server	Answer value	Client sends answer to server
GAME_RESULT	Server → Client	JSON object / text	Server sends result of answer evaluation
GAME_SCORES	Server → Client	JSON object	Server broadcasts current game scores

Connection Management

Table 3: protocol connection messages

Message Type	Direction	Payload Format	Description
PING	Both ways	(empty)	Ping message to measure latency
PONG	Both ways	(empty)	Response to ping message
DISCONNECT	Both ways	[message]	Notification of disconnection

A.3 Experiments Outputs

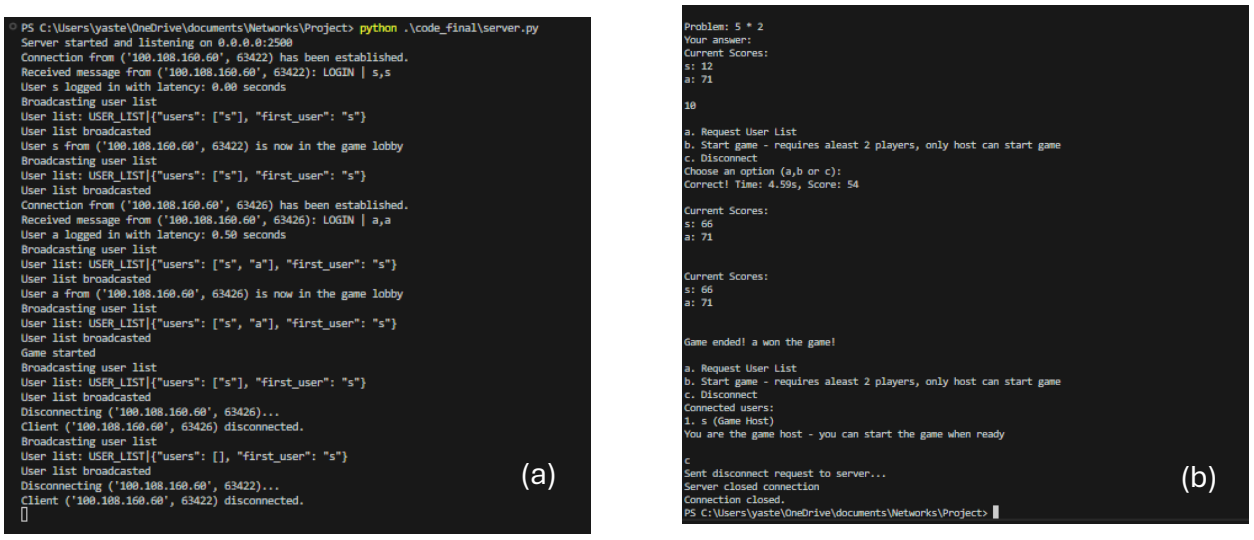

Figure 1: terminal output of server (a) and client (b) displaying latency measurement, game interaction and graceful disconnection

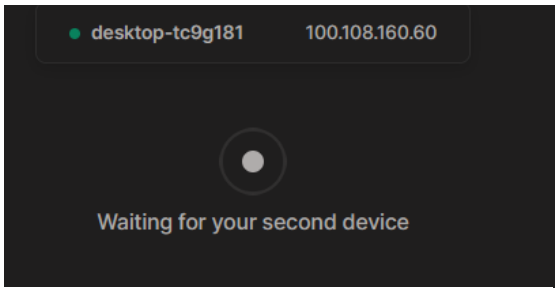

Figure 2: showing Tailscale configuration

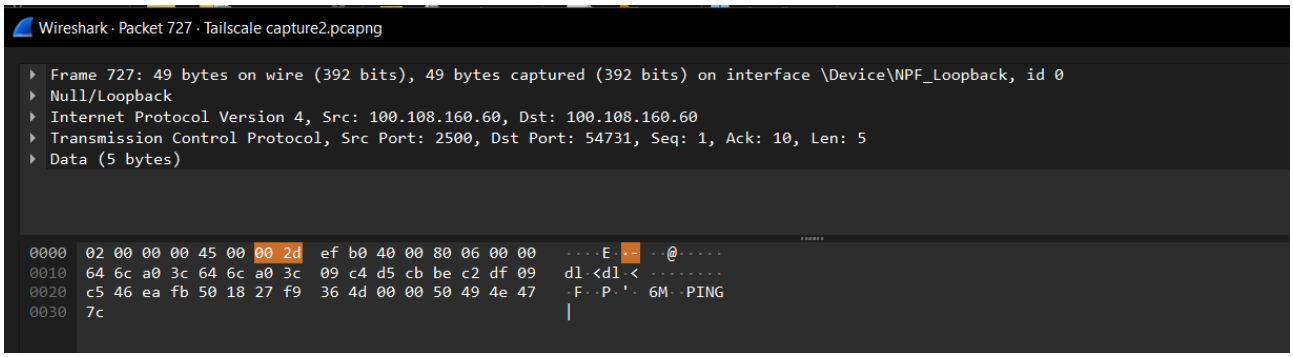

Figure 3: Wireshark recording of server ping request

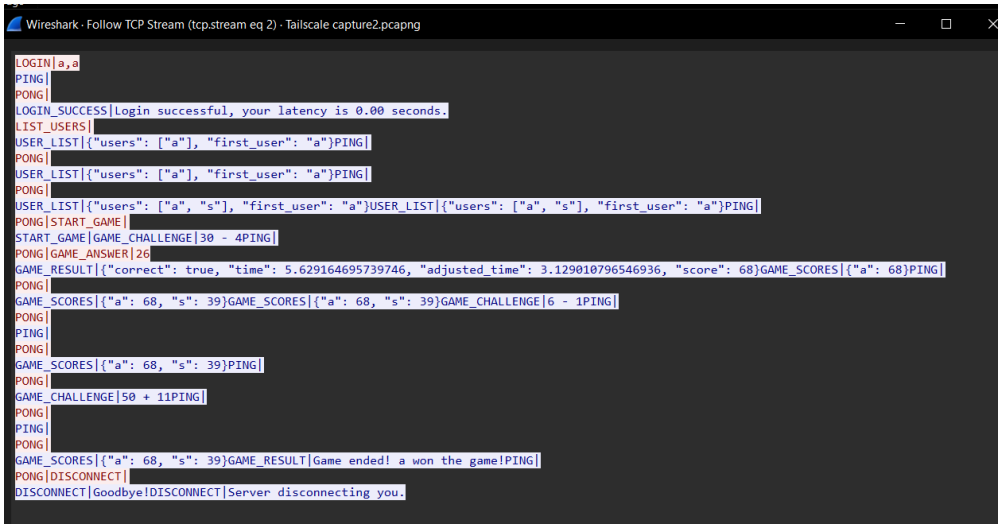

Figure 4: Wireshark recording of TCP chain of server client communication